



(All rights reserved)

UNIVERSITY OF GHANA LEGON
DEPARTMENT OF COMPUTER ENGINEERING
SCHOOL OF ENGINEERING SCIENCES
COLLEGE OF BASIC AND APPLIED SCIENCES

ARCHIVE 233

A Curated Secondhand and Vintage Menswear E-Commerce Platform

CPEN 208 —Software Engineering Project 3

Group 2

Romel Alvin Nii Lartey Lartey – 22263922
Cyril Desmond Ofori – 22393520
Edward Kakra Ankrah – 22382302
Owusu Koranteng Yaw Poku – 22263126
Kingsley Caldicock Quartey – 22407837

Sandra Naa Adaku Mettle – 22401641
Jude Gyampoh Addo – 22385323
Paula Akosua Asiedua Frimpong – 22381702
Kessey Ntiako David – 22299189
Reuben Adomako – 22377537

Repository: github.com/iamromelalvin7/swe_project3

Application: <https://archive233.vercel.app>

API: <https://archive233-backend.onrender.com>

Contents

Contents.....	2
1. Introduction	4
2. Problem statement.....	4
2.1 Context	4
2.2 Identified failures	4
2.3 Problem definition	4
3. Objectives.....	4
4. Requirements specification.....	5
4.1 Representative requirements.....	5
5. System analysis and design	5
5.1 Actors.....	6
5.2 Architecture	6
5.3 Layering and package structure	6
5.4 Design decision: availability as a derived value	6
5.5 Design decision: reservation holds with bounded lifetime	7
5.6 Design decision: authorisation without Row Level Security.....	7
5.7 Order lifecycle.....	7
6. Database design	7
6.1 Conventions.....	7
6.2 Entity relationship diagram.....	8
6.3 Tables.....	8
6.4 Normalisation.....	8
6.5 Concurrency statements	9
6.6 Indexing	9
6.7 Seed data	9
7. API design	9
7.1 Conventions.....	9
7.2 Error contract.....	9
7.3 Endpoints	10
7.4 Separate health endpoints.....	10
7.5 Pagination and authentication.....	10
8. Implementation.....	11
8.1 Concurrency control at checkout	11
8.2 Concurrency control at reservation	11
8.3 Image processing	11
8.4 Payment integration	12
9. User interface.....	12
10. Technologies used	17
10.1 Notable selections.....	17
11. Testing and verification	17
11.1 Concurrency (NFR-R2).....	17
11.2 Authorisation	17

11.3 Expiry independence (NFR-R8)	18
11.4 Performance	18
11.5 Responsiveness and interface states	18
11.6 Automated tests	18
12. Challenges encountered	18
12.1 Concurrency correctness	18
12.2 Payment confirmation failing silently	18
12.3 Cross-origin requests blocked on every authenticated route	19
12.4 Null parameter type inference	19
12.5 Oversized images in the catalogue grid	19
12.6 An overly broad ignore pattern	19
12.7 Scope management	19
13. Known limitations	19
14. Conclusion and future work	20
14.1 Conclusion	20
14.2 Future work	20
15. References	Error! Bookmark not defined.
Appendix A — Repository structure	21
Appendix B — Environment variables	21
Appendix C — Local setup	Error! Bookmark not defined.

1. Introduction

ARCHIVE 233 is a full-stack e-commerce platform developed for a single curated secondhand and vintage menswear retailer in Accra, Ghana. It comprises a Next.js client, a Java Spring Boot REST API, and a PostgreSQL database, deployed to production and integrated with the Paystack payment gateway.

The name refers to the archival nature of curated vintage clothing, in which each garment is an individually catalogued item rather than mass-produced stock, and to Ghana's international dialling code.

This report documents the problem addressed, the requirements derived from it, the design decisions taken, the implementation, the verification performed, and the defects encountered and resolved during development.

2. Problem statement

2.1 Context

Ghana is among the world's largest destinations for imported secondhand clothing. Kantamanto market in central Accra redistributes very large volumes of used garments weekly. Around this trade operates a tier of curated resellers who source selectively from the market, clean and photograph chosen pieces, and resell them at a margin to a younger urban customer base.

These resellers transact almost entirely through Instagram and WhatsApp: a garment is photographed and posted with its size and price, interested buyers send direct messages, the item is informally awarded to whoever is judged to have messaged first, payment is arranged by Mobile Money, and delivery is negotiated by voice call.

The workflow requires no software and incurs no platform fees, which explains its prevalence. It nevertheless fails in four specific ways.

2.2 Identified failures

FAILURE	DESCRIPTION
Double-selling	When several messages arrive within a short interval the seller has no reliable ordering and no mechanism for provisional allocation. Garments are promised twice. Because inventory is predominantly one-of-a-kind, no substitute can be offered.
No catalogue	A buyer seeking a particular size must scroll a chronological feed containing every item ever posted, sold or unsold. There is no filtering, no search, and no way to restrict results to available items.
No records	Order details exist only in chat history. Revenue over a period, best-selling categories, and outstanding orders cannot be determined. Disputes have no independent evidence.
No availability signal	A post from three weeks ago is visually identical whether the item sold within hours or remains unsold. The information exists but is not represented.

2.3 Problem definition

The core technical problem is not catalogue management, which is conventional record-keeping. It is the allocation of scarce, predominantly unique inventory under concurrent demand: ensuring that when multiple customers seek the same garment simultaneously, exactly one obtains it, and that this holds independently of application-level timing. This distinction shaped every subsequent design decision.

3. Objectives

#	OBJECTIVE
1	Provide the shop owner with a single system for listing inventory, tracking orders, and fulfilling deliveries.
2	Guarantee that inventory cannot be oversold, including under simultaneous checkout.
3	Provide customers an accurate, continuously current view of which items are available, reserved, and sold.

#	OBJECTIVE
4	Support the payment and delivery conventions in actual use in Ghana: Mobile Money, cash on delivery, and zone-based delivery pricing in cedis.
5	Demonstrate correct layered architecture, normalised relational design, and RESTful API practice.

4. Requirements specification

Requirements were specified in full before implementation began, each assigned a traceable identifier so that commits, tests, and this report may reference them unambiguously. Priorities follow the MoSCoW convention. Scope was frozen at the MUST tier; capabilities assessed as valuable but non-essential were documented as deferred rather than abandoned.

CATEGORY	MUST	SHOULD	COULD	TOTAL
Functional	61	16	2	79
Non-functional	40	6	—	46
Total specified	101	22	2	125
Explicitly excluded	—	—	—	11

The complete specification accompanies this report. A representative selection follows.

4.1 Representative requirements

ID	REQUIREMENT	PRI.
FR-09	The system shall display a paginated catalogue of published products.	MUST
FR-13	The system shall derive availability at the time of each read and shall not store availability as a persisted attribute.	MUST
FR-25	The system shall create a hold on the requested quantity when a customer adds a product to their cart, expiring a configurable interval after creation.	MUST
FR-28	The system shall exclude expired holds from availability calculation at read time, and shall not depend on any scheduled process for the correctness of that calculation.	MUST
FR-40	The system shall reduce stock by a conditional update guarded by a sufficiency predicate, and shall abort with a conflict response where no row is affected.	MUST
FR-41	The system shall record product title, size, image reference, and unit price on each order line as a snapshot taken at purchase.	MUST
FR-45	The system shall confirm payment solely by a server-initiated verification request to the payment provider.	MUST
NFR-12	The system shall verify, for every operation on a specific record, that the acting user is permitted to act upon that record.	MUST
NFR-20	Stock quantity shall never fall below zero; enforced by a database constraint independent of application logic.	MUST
NFR-21	Where two customers attempt to purchase the final unit simultaneously, exactly one shall succeed and the other shall receive a conflict response.	MUST
NFR-22	Monetary values shall be stored as integers denominated in pesewas; floating-point shall not be used for currency.	MUST
NFR-27	The correctness of system state shall not depend upon the execution of any scheduled background process.	MUST

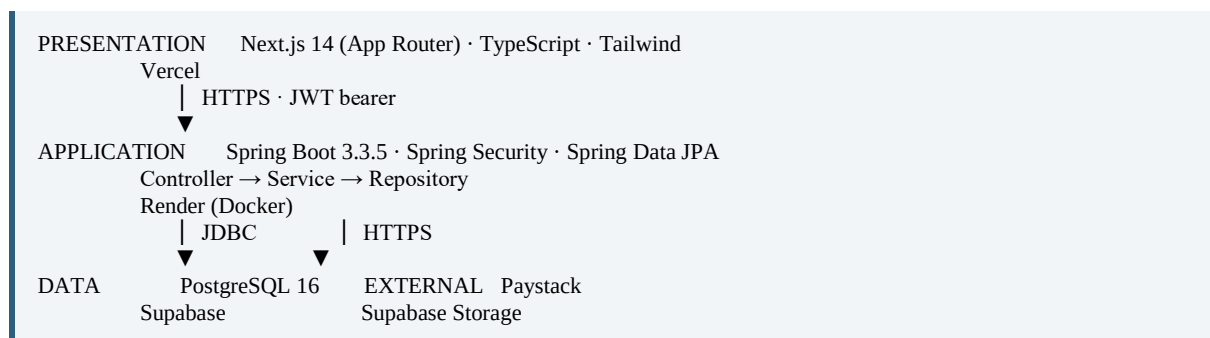
5. System analysis and design

5.1 Actors

ACTOR	CAPABILITIES
Visitor	Browse, search, filter, view product detail
Customer	All visitor capabilities, plus cart, checkout, and order history
Administrator	Product management, order management, dashboard

The system serves exactly one retail operation. Multi-vendor operation was excluded during analysis: supporting independent sellers introduces multi-tenancy, a cross-cutting concern affecting every query and authorisation check in the system.

5.2 Architecture



The defining property of this architecture is what it omits: the browser holds no database credential and issues no query. All access is mediated by the application tier, which is the sole credential holder and therefore the sole point at which authorisation can be enforced.

5.3 Layering and package structure

LAYER	RESPONSIBILITY	PROHIBITED
Controller	HTTP concerns, deserialisation, input validation, status codes	Business rules
Service	Business rules, transaction boundaries, orchestration	HTTP types, data access
Repository	Data access	Business rules

The backend is organised one package per bounded concern rather than by technical layer:

```
com.archive233.backend
  auth · user · catalog · cart · order · payment · delivery
  storage · error · config · health · common
```

A single `@RestControllerAdvice` emits the one error contract across every endpoint.

5.4 Design decision: availability as a derived value

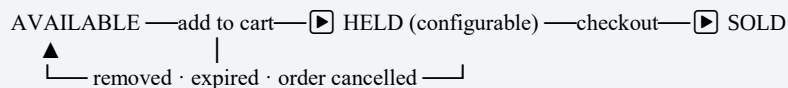
The intuitive model assigns each product a boolean indicating availability. This was rejected. Availability is not an intrinsic property of a product; it is a function of stock held and reservations currently active, and it changes without any write to the product record — specifically, at the moment a reservation lapses. A stored boolean would require a background process to remain accurate and would be incorrect until that process ran. Correctness would depend on scheduling.

Availability is instead computed at each read as stock quantity less the sum of live holds, exposed as a database view so every consumer computes it identically. A lapsed hold is excluded by its own predicate, so the value is correct irrespective of whether any background process has executed.

5.5 Design decision: reservation holds with bounded lifetime

Conventional e-commerce treats a cart as a non-binding selection. This was assessed as inappropriate here for two reasons: inventory is predominantly single-quantity, so contention is the normal case rather than an exception; and the workflow being replaced already treats an expression of intent as binding, so a system behaving otherwise would be experienced as a regression.

An unbounded hold would permit one inactive user to render the catalogue unavailable — customers do not empty carts, they close tabs. The hold is therefore bounded, with its duration supplied by configuration rather than embedded as a constant.



5.6 Design decision: authorisation without Row Level Security

PostgreSQL Row Level Security was considered and deliberately not adopted. RLS is the appropriate control where a client holds a database credential and queries directly. Here the browser communicates only with the application tier, which holds the sole credential; authorisation is enforced in the service layer per resource, verifying that the acting user may act upon the specific record concerned rather than merely that they are authenticated.

Introducing RLS would duplicate this logic in a second location with different semantics, creating a maintenance burden and risk of divergence for no additional protection. A partially implemented policy would be worse than none, creating an impression of defence in depth that did not exist.

5.7 Order lifecycle



Transitions are enforced in the service layer; those not permitted by the machine are rejected with 409 rather than silently ignored. DELIVERED is terminal. Cancellation returns ordered quantities to available stock.

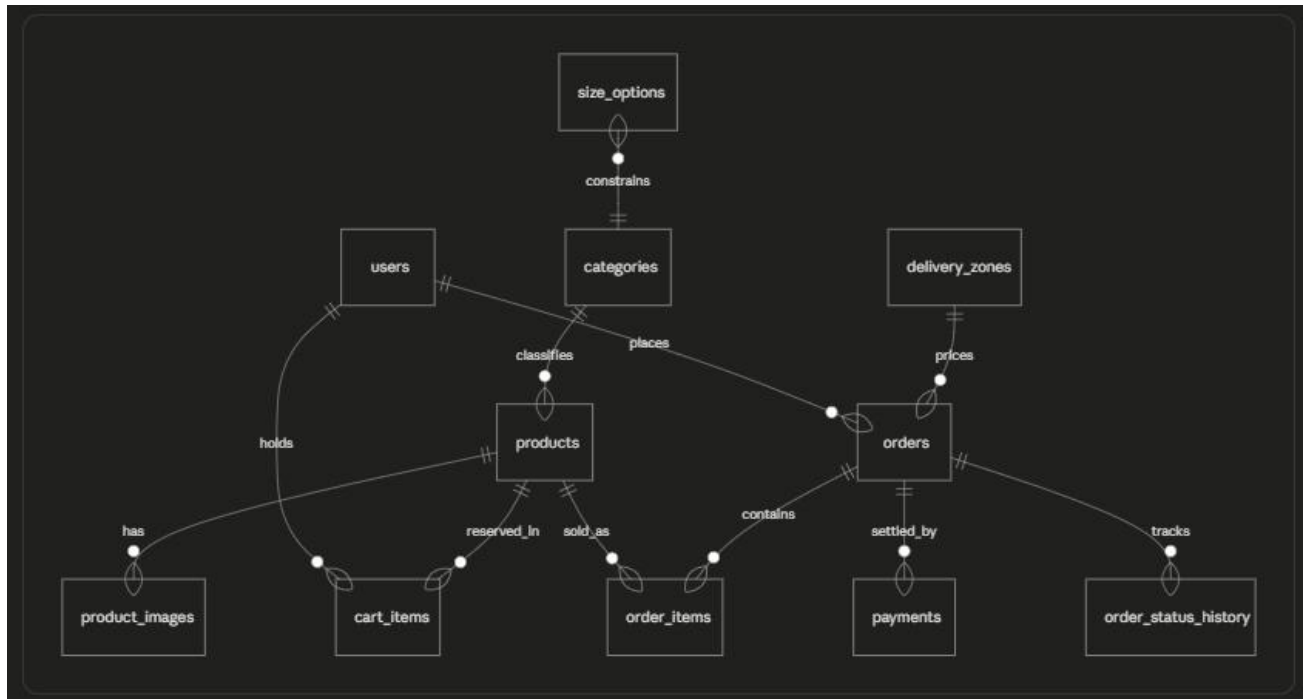
6. Database design

The schema is owned entirely by db/01_schema.sql. Hibernate is configured to validate the schema and never to generate or alter it. Eleven tables and one view.

6.1 Conventions

CONVENTION	RATIONALE
UUID primary keys	Not enumerable; prevents inference of record counts and traversal by increment
Money as integer pesewas	Binary floating point cannot represent decimal fractions exactly; rounding error in financial calculation accumulates
timestampz, stored UTC	A timestamp without zone is ambiguous once any component runs in a different region
CHECK constraints for domain rules	Invariants the database can enforce are enforced there, independently of application correctness

6.2 Entity relationship diagram



6.3 Tables

TABLE	PURPOSE AND PRINCIPAL CONSTRAINTS
users	Accounts and roles. Role constrained to CUSTOMER or ADMIN; unique email; carries a default delivery address.
categories	Product classification. Each carries a size group: APPAREL, WAIST, FOOTWEAR, or ONE_SIZE.
size_options	Controlled vocabulary of size labels per size group. Size is never free text, specifically to prevent the L / 1 / Large / large drift that breaks size filtering.
products	Inventory. Status (DRAFT, PUBLISHED, ARCHIVED) expresses owner intent and is entirely separate from availability. stock_quantity carries CHECK (≥ 0); price_pesewas carries CHECK (> 0).
product_images	Stores both a display derivative and a thumbnail for each image; unique display position per product.
delivery_zones	Zone-based delivery fees in pesewas rather than a flat national rate.
cart_items	This table is the reservation-hold mechanism. Composite primary key on user and product; expiry evaluated at read time, not by a job.
orders	Immutable historical records. Delivery fields snapshotted at checkout, never re-joined to live users.
order_items	Line items with title, size, image reference, and unit price snapshotted at purchase.
order_status_history	State transition audit trail, timestamped and attributed.
payments	Payment records. provider_reference is UNIQUE, so a replayed webhook cannot double-insert.
product_availability	View. The single definition of availability: stock quantity less the sum of live holds.

6.4 Normalisation

The schema is in third normal form. Two apparent violations are deliberate denormalisations.

Order line snapshots. `order_items` stores product title, size, image reference, and unit price, all also present on products. An order is a historical record of an agreed transaction; rendering it by joining to the live table would cause a subsequent price amendment to silently rewrite past receipts.

Delivery detail snapshots. `orders` stores recipient name, telephone, address, zone name, and fee, duplicating users and `delivery_zones`. A customer who later updates their profile address must not thereby alter the record of where a previous order was despatched.

These are not normalisation failures. A live product and a purchased line item are genuinely distinct entities that happen to share attribute names.

6.5 Concurrency statements

Two reference statements at the foot of the schema file carry the entire concurrency guarantee and are used verbatim in application code rather than paraphrased. They are discussed in section 8.

6.6 Indexing

Indexes exist on every foreign key and every filtered or sorted column: product status combined with creation date, category, brand lowercased for case-insensitive matching, price, and size label; orders by user and by status, each with creation date; and a composite index on `cart_items` covering product and expiry to support the availability aggregation.

6.7 Seed data

DATASET	CONTENTS
<code>db/02_seed.sql</code>	Categories, size options per size group, delivery zones, and one administrator account
<code>db/03_seed_orders.sql</code>	32 historical orders spanning every lifecycle status, dated across approximately 58 days
Products	15 products, each with genuine photography

The specification targeted 40 seeded products. Fifteen were sourced. Padding the catalogue with unrelated stock imagery was rejected: seed data that misrepresents the shop's actual inventory would undermine the purpose of the demonstration. The shortfall is recorded as a known gap rather than concealed.

7. API design

7.1 Conventions

Resources are addressed by noun, operations expressed by HTTP verb. Status codes carry meaning: 200 retrieval, 201 creation, 400 malformed input, 401 absent or invalid authentication, 403 insufficient permission, 404 absent resource, 409 conflict with current state. The distinction between 401 and 403 is observed strictly. The API never returns a success status accompanied by an error payload.

7.2 Error contract

```
{
  "error": {
    "code": "OUT_OF_STOCK",
    "message": "Only 1 unit of this item remains.",
    "fields": { "quantity": "Requested 3, available 1" }
  }
}
```

The code is machine-readable and stable, allowing the client to branch without parsing prose. The fields object appears only on validation failure and maps input names to messages, which is what permits errors to render adjacent to the input concerned rather than as an undifferentiated banner.

7.3 Endpoints

METHOD	PATH	AUTH	OK	ERRORS
POST	/api/auth/register	—	201	400, 409
POST	/api/auth/login	—	200	400, 401
GET	/api/products	—	200	400
GET	/api/products/{id}	—	200	404
GET	/api/categories	—	200	—
GET	/api/delivery-zones	—	200	—
GET	/api/cart	Customer	200	401
POST	/api/cart/items	Customer	201	400, 401, 409
PATCH	/api/cart/items/{productId}	Customer	200	400, 401, 409
DELETE	/api/cart/items/{productId}	Customer	204	401, 404
POST	/api/checkout	Customer	201	400, 401, 409
GET	/api/checkout/verify	Customer	200	400, 401, 404
POST	/api/webhooks/paystack	Signature	200	400
GET	/api/orders	Customer	200	401
GET	/api/orders/{id}	Customer	200	401, 403, 404
POST	/api/orders/{id}/cancel	Customer	200	401, 403, 409
GET	/api/admin/products	Admin	200	401, 403
POST	/api/admin/products	Admin	201	400, 401, 403
PUT	/api/admin/products/{id}	Admin	200	400, 401, 403, 404
POST	/api/admin/products/{id}/images	Admin	201	400, 401, 403, 413
POST	/api/admin/products/{id}/archive	Admin	200	401, 403, 404
GET	/api/admin/orders	Admin	200	401, 403
GET	/api/admin/orders/{id}	Admin	200	401, 403, 404
PATCH	/api/admin/orders/{id}/status	Admin	200	400, 401, 403, 409
GET	/api/admin/dashboard	Admin	200	401, 403
GET	/api/health	—	200	—
GET	/api/health/db	—	200	503

7.4 Separate health endpoints

Two health endpoints are provided. The first performs no database access and supports frequent uptime monitoring; the second executes a trivial query and reports genuine readiness. A single combined endpoint would render monitoring unnecessarily expensive or readiness checks meaningless.

7.5 Pagination and authentication

Every collection endpoint paginates, returning twenty records by default and no more than fifty. An unpaginated collection endpoint is a latent failure that manifests only once data volume grows, and was therefore treated as a requirement rather than an optimisation.

Login returns a JSON Web Token signed with HMAC-SHA256, carrying subject and role claims with an expiry. A servlet filter validates the signature on each request and populates the security context; authorisation is then checked per resource within service methods.

8. Implementation

8.1 Concurrency control at checkout

The central correctness requirement is that concurrent purchases cannot oversell. The naive implementation does not satisfy it:

```
Product p = repository.findById(productId);

if (p.getStockQuantity() >= quantity) {    // both threads read 1
    p.setStockQuantity(
        p.getStockQuantity() - quantity); // both compute 0
    repository.save(p);                  // both write 0
}
// two orders created against one unit of stock
```

Two threads may each read a stock quantity of one, each evaluate the condition as true, and each write zero. The defect is invisible under sequential testing because it requires interleaved execution to manifest.

The implemented approach is a single conditional statement:

```
UPDATE products
SET stock_quantity = stock_quantity - :qty
WHERE id = :pid
AND stock_quantity >= :qty;
```

PostgreSQL evaluates the predicate and performs the write as one atomic operation under row-level locking. There is no interval between evaluation and write during which another transaction may intervene. Where zero rows are affected, another transaction has already claimed the stock; the enclosing transaction is rolled back and the request answered with 409. A CHECK constraint provides a final guarantee independent of application correctness.

8.2 Concurrency control at reservation

Claiming a hold uses a different technique, because the sufficiency check involves an aggregate over a second table and therefore cannot be expressed as a single conditional statement. A row lock is acquired on the product for the duration of the check and insert.

```
SELECT stock_quantity FROM products
WHERE id = :pid FOR UPDATE;    -- serialises concurrent claims

-- compare requested quantity against stock less live holds

INSERT INTO cart_items (user_id, product_id, quantity, expires_at)
VALUES (:uid, :pid, :qty, now() + :ttl)
ON CONFLICT (user_id, product_id) DO UPDATE
SET quantity = :qty, expires_at = now() + :ttl;
```

The system therefore employs pessimistic locking where the check requires an aggregate across tables, and optimistic concurrency control where the new value is derivable from the old. Each technique is applied where its trade-offs are favourable rather than uniformly.

8.3 Image processing

Image resizing was relocated from server to client. Decoding a typical mobile photograph into an uncompressed bitmap requires approximately 48 megabytes of heap; the production instance provides 512 megabytes in total, and concurrent uploads exhausted it during development.

The browser downscales and re-encodes before transmission, which also removes embedded metadata — including location coordinates on mobile photographs — as a consequence of drawing to a canvas element. The server validates file type by inspection of leading bytes rather than trusting the declared content type, derives a thumbnail, and stores two derivatives.

8.4 Payment integration

Payment is initialised server-side with the order total denominated in pesewas and a unique reference. The customer completes payment on the provider's hosted page and is redirected back.

The redirect is not treated as confirmation. It is a browser navigation and may be fabricated by any party. Confirmation requires a server-to-server verification request issued with the secret key, and only that response is acted upon. A webhook endpoint provides backstop confirmation for customers who close the browser after paying; it verifies the HMAC signature over the raw request body before processing and is idempotent, so repeated delivery of the same event produces no state change beyond the first.

9. User interface

The interface follows a restrained editorial aesthetic appropriate to curated garments, in which photography carries the visual weight and typography and whitespace provide structure. It is designed mobile-first, on the basis that customers access the platform predominantly from mobile devices over metered connections.

1. Store Catalogue

Archive 233

Q

CATEGORY ▾

SIZE ▾

BRAND ▾

CONDITION ▾

PRICE ▾

SORT: NEWEST ▾

24 PIECES



NIKE · US 10 · FOOTWEAR
Nike SB Dunks - Low
GH¢ 350.00
1 LEFT



NIKE · US 9 · FOOTWEAR
Jordan 1 - Low
GH¢ 300.00



UNBRANDED · L · SHIRTS
Casual Polo
GH¢ 85.00
1 LEFT



RALPH LAUREN · L · SHIRTS
Polo Ralph Lauren
GH¢ 150.00

Product Detail Page

Archive 233

← BACK TO CATALOG





Nike SB Dunks - Low

NIKE · US 10 · EXCELLENT

GH¢ 350.00

1 AVAILABLE

ADD TO CART

Adding to cart reserves this item for 10 minutes.

BRAND	Nike
CATEGORY	Footwear
SIZE	US 10
CONDITION	EXCELLENT
COLOUR	Orange

Cart

Cart



Nike SB Dunks - Low

NIKE · US 10

RESERVED FOR YOU · 08:53

- 1 +

Remove

GH¢ 350.00



Jordan 4 High Top

NIKE · US 8

RESERVED FOR YOU · 09:16

- 1 +

Remove

GH¢ 350.00



Retro New York Varsity Jacket

UNBRANDED · XL

RESERVED FOR YOU · 09:26

- 1 +

Remove

GH¢ 250.00

SUMMARY

Subtotal

GH¢ 950.00

Delivery

At checkout

Total

GH¢ 950.00

CHECKOUT

Checkout page

← RETURN TO CART

Checkout

FULL NAME

Romel Lartey

PHONE NUMBER

0200819543

DELIVERY ADDRESS

Kampala Courts, Pent Hostel

DELIVERY ZONE

Madina · GH¢ 30.00

PAYMENT METHOD

Mobile Money / Card — via Paystack

Cash on delivery

ORDER SUMMARY

Nike SB Dunks - Low

NIKE · US 10 × 1

GH¢ 350.00

Jordan 4 High Top

NIKE · US 8 × 1

GH¢ 350.00

Retro New York Varsity Jacket

UNBRANDED · XL × 1

GH¢ 250.00

Subtotal

GH¢ 950.00

Delivery fee

GH¢ 30.00

Total

GH¢ 980.00

PAY GH¢ 980.00

PROCESSED SECURELY VIA PAYSTACK

Confirmation page

ORDER CONFIRMED

Thank you

We have your order and the pieces are held for you. Delivery is arranged by phone within a working day.

ORDER NUMBER

AR-R3BPB2DC

DELIVERING TO

Romel Lartey

PHONE

0200819543

ADDRESS

Kampala Courts, Pent Hostel · Madina

PAYMENT

Mobile Money / Card

TOTAL PAID

GH¢ 980.00

ITEMS



Nike SB Dunks - Low

US 10

Qty 1

GH¢ 350.00



Jordan 4 High Top

Customer Order History

Archive 233

Account

DETAILS

MY ORDERS

AR-4BBMSAYD	17 AUG 2026	1 item	GH¢ 180.00	PENDING
AR-GKYQGAAW	16 AUG 2026	1 item	GH¢ 440.00	CONFIRMED
AR-P8TE9BZ6	16 AUG 2026	1 item	GH¢ 130.00	CONFIRMED
AR-URCEVPV3	16 AUG 2026	1 item	GH¢ 140.00	DELIVERED
AR-K98J74NG	16 AUG 2026	1 item	GH¢ 130.00	CONFIRMED
AR-EEFEPTYU	16 AUG 2026	2 items	GH¢ 560.00	CONFIRMED
AR-3MK58UCC	16 AUG 2026	1 item	GH¢ 120.00	CANCELLED

Admin dashboard

Archive 233

DASHBOARD

DASHBOARD

PRODUCTS

ORDERS

Overview

TOTAL REVENUE	ORDERS	ITEMS SOLD	LIVE STOCK
GH¢ 5,240.00	48	26	10

Awaiting action

ORDER	CUSTOMER	PHONE	ZONE	ITEMS	TOTAL	PAYMENT	STATUS
AR-UGRHRVGT	Ama Serwaa	0201234502	Accra Central	2	GH¢ 230.00	PAID	OUT FOR DELIVERY
AR-BFVA4TVX	Kojo Mensah	0201234503	East Legon	1	GH¢ 80.00	PENDING	OUT FOR DELIVERY
AR-JLNMGG9N	Abena Boateng	0201234504	Tema	1	GH¢ 200.00	PAID	OUT FOR DELIVERY
AR-9W45CZE3	Yaw Asante	0201234505	Madina	2	GH¢ 420.00	PENDING	PACKED

Products creation page – Admin side

DASHBOARD

PRODUCTS

ORDERS

New product

Drop photos

MAX 6

1 IS PRIMARY

TITLE

e.g. 1990s Carhartt Detroit Jacket

CATEGORY

Select

▼

BRAND

Carhartt

SIZE

Pick a category first

▼

CONDITION

Select

▼

PRICE GHS

0.00

QUANTITY

1

Leave at 1 for one-of-a-kind pieces

COLOUR

Hamilton Brown

ERA

1990s

FIT NOTES

How does this piece measure and drape?

FLAWS

Any notable wear, tear or discolouration

Order Processing Page – Admin side

← AR-R3BPB2DC

CONFIRMED

DELIVERY

NAME

Romel Larley

PHONE

0200819543

ADDRESS

Kampala Courts, Pent Hostel

ZONE

Madina

PLACED

23/08/2026, 23:12:16

ITEMS

3 ITEMS



Nike SB Dunks - Low

US 10

Qty 1

GH¢ 350.00



Jordan 4 High Top

US 8

Qty 1

GH¢ 350.00



Subtotal

GH¢ 950.00

Delivery fee

GH¢ 30.00

Total

GH¢ 980.00

PAYMENT

MOBILE MONEY · PAYSTACK

STATUS

PENDING → CONFIRMED → PACKED →

OUT FOR DELIVERY → DELIVERED

MARK AS PACKED

CANCEL ORDER

10. Technologies used

LAYER	TECHNOLOGY	JUSTIFICATION
Client	Next.js 14 (App Router), TypeScript, Tailwind	Server components reduce JavaScript delivered to the browser, which matters on metered connections; types carry across the API boundary
Server	Spring Boot 3.3.5, Java 21	Layered structure supported by framework conventions; long-term support release
Security	Spring Security with JWT	Role-based access control
Persistence	Spring Data JPA / Hibernate	Configured ddl-auto=validate; validates the schema, never modifies it
Database	PostgreSQL 16	Views, CHECK constraints, and atomic conditional writes
DB hosting	Supabase	Managed PostgreSQL compatible with standard tooling
Object storage	Supabase Storage	Images stored outside the database
Local environment	Docker Compose	Identical schema across machines; functions without an internet connection
Payments	Paystack, test mode	Operates in Ghana, denominates in cedis, accepts amounts in pesewas
Client hosting	Vercel	First-party support for the client framework
Server hosting	Render, Docker	Container control permitting JVM tuning within a 512 MB instance
Version control	Git and GitHub	Conventional commit messages

10.1 Notable selections

Docker rather than the native Java environment. The instance provides 512 megabytes of memory. A container permits explicit configuration of heap sizing, garbage collector selection, and just-in-time compilation tiering — the difference between stable operation and termination on memory exhaustion.

Supabase for PostgreSQL only. The platform's authentication service was not adopted. A second authentication system operating alongside Spring Security would duplicate the role model and expand the attack surface without benefit.

11. Testing and verification

Development proceeded through phase gates. Each phase concluded with a checkpoint requiring demonstrable evidence — terminal output, request transcripts, or reachable deployed URLs — before the next commenced. Phases 1 through 3 are complete and verified; Phase 4 is partially complete, as recorded in section 11.6.

11.1 Concurrency (NFR-R2)

The central correctness claim was verified directly. With a product configured at a stock quantity of one, two simultaneous POST /api/checkout requests were issued. The result was exactly one 201 with an order created, one 409 carrying OUT_OF_STOCK, and a resulting stock quantity of zero. The test was performed twice: once against the local environment and once against the deployed production URL.

11.2 Authorisation

Administrative routes were confirmed to return 403 when presented with a token issued to a Customer account. Order retrieval was confirmed to be scoped to the owning customer. Cross-origin requests were confirmed restricted to the configured production origin: a Vercel preview URL is correctly refused, which is the intended behaviour rather than a defect.

11.3 Expiry independence (NFR-R8)

With the scheduled housekeeping process disabled, a reservation was permitted to lapse and availability re-queried. The lapsed hold was correctly excluded from the computation, confirming that correctness does not depend upon the background process.

11.4 Performance

METRIC	RESULT
Lighthouse Performance (mobile)	89
Lighthouse Accessibility	95
Lighthouse Best Practices	100
Lighthouse SEO	100
Largest Contentful Paint	3.6 s
Catalogue page weight	651 KiB (from 3.98 MB before thumbnail correction)
Query count per catalogue listing (NFR-P5)	One, irrespective of row count

LCP remains above the 2.5 second threshold specified in NFR-04. This is attributed to cold time-to-first-byte on the free-tier server instance and image CDN latency under simulated fourth-generation conditions. Scheduled health-check pinging is intended to mitigate the former and has not yet been configured.

11.5 Responsiveness and interface states

The interface was swept at 360 pixels using viewport emulation, which surfaced one genuine overflow defect in the administrative dashboard metric grid — a non-wrapping layout without a minimum-width reset, which broke on realistic cedi figures that the design mockups never exercised. Physical device testing was not performed as no device was available. Loading, empty, and error states were verified on every list by forcing server errors.

11.6 Automated tests

State the actual position here before submission.

The suite currently includes AuthApiTest, which executes against the live database rather than an isolated test schema and fails because a seeded customer email collides with its hardcoded duplicate-email fixture. This is not a code regression but it does demonstrate that the suite is fragile to production data. Record the full set of tests written, the components covered, and this limitation. An accurate account of limited automated coverage alongside the substantial manual and Playwright verification described above is credible; an unsupported claim of comprehensive testing is trivially falsified by inspecting the repository.

12. Challenges encountered

The defects below were found by running the application against live data rather than by reading code. Each is recorded with its root cause and resolution.

12.1 Concurrency correctness

The initial checkout implementation checked stock in the service layer before decrementing. This is correct under sequential access and incorrect under concurrent access, and the defect is invisible in single-user testing because it requires interleaved execution to manifest. It was identified by reasoning about the possible interleavings rather than by observing a failure.

The resolution relocated the guarantee from application code into a single SQL statement whose predicate and write are atomic, supported by a database constraint as a final barrier. The general lesson is that guarantees required to hold under concurrency belong at the storage layer, not in application logic.

12.2 Payment confirmation failing silently

The most serious defect encountered. The verification and webhook handlers in the checkout service were missing transactional annotations, so lazy-loaded associations threw an initialisation exception on every call, deterministically. The consequence was that payments were completed at the provider but never recorded by the application and the associated orders never advanced.

The failure was compounded by the webhook backstop, which was swallowing the identical exception and returning a success status to the provider regardless — so the mechanism designed to catch the failure was concealing it. This is a general hazard of backstop handlers: an error path that reports success is worse than no error path at all.

The resolution added the missing transaction boundaries. Six affected transactions were then reconciled individually against the provider's own API: four were confirmed and their orders advanced, one proved to be a genuine decline, and one belonged to an order cancelled before reconciliation and was referred to the shop owner rather than resolved automatically.

12.3 Cross-origin requests blocked on every authenticated route

Cross-origin configuration was implemented as a web MVC configurator, which applies after the security filter chain's authorisation check. Preflight requests to any authenticated route were therefore rejected before cross-origin headers were added, making every authenticated route unreachable from a browser while remaining reachable from command-line tooling. Resolved by configuring cross-origin handling within the security configuration so that it executes first.

12.4 Null parameter type inference

The PostgreSQL JDBC driver could not infer the type of null bind parameters used only inside a function or cast, which occurred three separate times: once on a null search string passed to a lowercasing function, and twice on optional administrative order filters. Rather than continue patching individual queries, the administrative order search was rewritten using JPA Specifications, so an absent filter never becomes an ambiguous bind parameter at all.

12.5 Oversized images in the catalogue grid

The catalogue served display-sized derivatives where thumbnails were intended. Identified through performance auditing rather than functional testing. Correction reduced page weight approximately sixfold, raised the mobile performance score from 71 to 89, and reduced largest contentful paint from 7.1 to 3.6 seconds.

12.6 An overly broad ignore pattern

A pattern intended to exclude credential files from version control was matching a source file whose name contained the same substring, silently excluding it from every commit. Local builds succeeded because the file remained on disk; the omission surfaced only when the deployment platform performed a clean clone and the build failed. The episode illustrates that a build succeeding locally is not evidence that the repository is complete.

12.7 Scope management

The schedule created continuous pressure to add capabilities. This was managed by freezing scope at the MUST tier before implementation, maintaining a gated queue of enhancements that could not be started until all mandatory requirements had passed, and recording rejected proposals rather than discarding them. The gated queue remains untouched.

13. Known limitations

The following are recorded rather than concealed. Each was identified during verification and logged for resolution.

LIMITATION	DETAIL
Order line thumbnails	The order snapshot stores the display-sized image reference rather than the thumbnail — the same class of defect corrected in the catalogue grid, not yet applied to the snapshot path.
Unhandled fetch rejections	Two detail views issue requests without a rejection handler, so a failed request leaves the view in a permanent loading state.

LIMITATION	DETAIL
Test isolation	The authentication test suite executes against the live database rather than an isolated schema and is therefore sensitive to seeded data.
WebP upload	WebP files pass content-type validation but fail during processing, as no decoder for that format is present on the classpath. The accepted-format claim and the actual capability disagree.
Catalogue size	Fifteen products are seeded against a specified target of forty.

14. Conclusion and future work

14.1 Conclusion

ARCHIVE 233 implements a complete e-commerce workflow for a single-property secondhand menswear retailer: catalogue browsing with filtering and search, reservation-based carts with bounded lifetime, transactional checkout with payment gateway integration, and administrative inventory and order management. The system is deployed and operational at the addresses given on the title page.

The engineering substance of the project is not the create, read, update, and delete layer, which is conventional. It is the allocation of scarce inventory under concurrent demand, and the decisions that follow from it: deriving availability rather than storing it, bounding reservations with lazily evaluated expiry, and placing the guarantee against overselling at the storage layer where it holds independently of application timing.

The project further demonstrates that a modest investment in design before implementation — a requirements specification with traceable identifiers, a schema designed in advance of entity classes, and explicitly documented scope boundaries — substantially reduces rework on a compressed schedule. It also demonstrates that the defects which matter most are found by running the system against real data, not by reading code: the most serious defect encountered was deterministic, affected every payment, and was concealed by the very mechanism designed to catch it.

The system is not production-ready, and this is stated deliberately rather than omitted. Payment reconciliation is manual, customer notification is absent, and no returns workflow exists. These were scoped out as decisions rather than overlooked.

14.2 Future work

CAPABILITY	ASSESSMENT
Analytics	Sales by category, revenue over time, brand performance. Aggregate queries over existing data; no schema change required. The highest-value remaining addition.
Customer wishlists	Saved items providing demand signal to the owner. One table, two endpoints.
Discount codes	Percentage and fixed-amount promotions at checkout. Touches the checkout transaction and would require re-verification of the concurrency guarantee.
Product reviews	Complicated by single-quantity inventory: an item sold once cannot accumulate reviews. Would require reviewing the seller rather than the product.
Notifications	Electronic mail and short message on status change. Requires a delivery provider and introduces deliverability concerns disproportionate to project scope.
Returns and refunds	A full workflow with payment reversal and stock reinstatement.
Real-time availability	Push notification of reservation changes to connected clients. High risk on suspending infrastructure; polling provides most of the benefit at a fraction of the risk.
Multi-vendor operation	Introduces multi-tenancy, a cross-cutting concern affecting every query and authorisation check. A rewrite rather than an extension.

Appendix A — Repository structure

```
swe_project3/
├── CLAUDE.md
├── docker-compose.yml
├── backend/      Spring Boot application
├── frontend/     Next.js application
├── db/
│   ├── 01_schema.sql
│   ├── 02_seed.sql
│   ├── 03_seed_orders.sql
│   └── backup.sql  pg_dump export
├── docs/         PRD, OBJECTIVES, design export
└── logs/         BUILD_LOG.md, SUGGESTIONS.md
```

Appendix B — Environment variables

The following keys are required. Values are supplied by environment configuration and are not committed; backend/.env.example documents the shape without values.

```
SPRING_DATASOURCE_URL      SUPABASE_URL
SPRING_DATASOURCE_USERNAME  SUPABASE_SERVICE_ROLE_KEY
SPRING_DATASOURCE_PASSWORD  SUPABASE_BUCKET
JWT_SECRET                  CORS_ALLOWED_ORIGINS
JWT_EXPIRY_HOURS            CART_HOLD_MINUTES
PAYSTACK_SECRET_KEY
PAYSTACK_BASE_URL
```